

Práctica Integradora: Montículos (Heaps) y Colas de Prioridad

1. Nombre de la práctica

Diseño, Implementación y Aplicación de Montículos en Problemas Reales

2. Objetivo general

Implementar y analizar estructuras de datos tipo **heap (montículos)** para resolver problemas reales que requieren manejo eficiente de prioridades.

3. Objetivos específicos

- Implementar montículos (min-heap y max-heap) desde cero.
- Aplicar operaciones fundamentales: inserción, extracción y reorganización.
- Analizar complejidad temporal en escenarios reales.
- Integrar heaps en algoritmos clásicos y sistemas prácticos.
- Evaluar su comportamiento en aplicaciones dinámicas.

4. Descripción de la práctica

El estudiante desarrollará cinco aplicaciones basadas en **montículos**, donde cada problema representa un caso real de uso de colas de prioridad.

Cada ejercicio deberá incluir:

- Implementación
- Evidencia de ejecución
- Análisis de comportamiento

5. Ejercicios

Ejercicio 1 — Sistema de Triage en Emergencias (Min-Heap)

Descripción

Simular un sistema hospitalario donde los pacientes se atienden según su nivel de gravedad.

Requisitos

- Implementar un **min-heap**
- Cada paciente tiene:
 - ID
 - Nivel de gravedad (1 = más crítico)

Operaciones

- Inserción de paciente → aplicar **sift-up**
- Atención de paciente → aplicar **extract-min + sift-down**

Evidencia

- Estado del heap después de cada operación
- Orden de atención

Análisis

- Justificación del uso de min-heap
- Complejidad de operaciones

Ejercicio 2 — Priorización en IA de videojuego (Max-Heap)

Descripción

Simular la lógica de producción de unidades en un sistema de estrategia.

Requisitos

- Implementar un **max-heap**
- Representación en arreglo
- Índices:
 - Hijo izquierdo: $(2i + 1)$
 - Hijo derecho: $(2i + 2)$

Actividad

- Insertar al menos 10 unidades con prioridad:
 - Defensores
 - Atacantes
 - Trabajadores

Evidencia

- Arreglo del heap en cada inserción
- Orden de extracción

Análisis

- Acceso a la raíz en $O(1)$
- Impacto de prioridades dinámicas

Ejercicio 3 — Algoritmo HeapSort

Descripción

Ordenar un arreglo utilizando montículos.

Requisitos

1. Construir heap con método **bottom-up**
2. Aplicar proceso de ordenamiento:
 - Intercambio raíz ↔ último elemento
 - Reducción del heap
 - Aplicación de **sift-down**

Evidencia

- Estado del arreglo en cada iteración
- Resultado final ordenado

Análisis

- Complejidad: $O(n \log n)$
- Comparación con otros algoritmos

Ejercicio 4 — Planificador de CPU (Min-Heap)

Descripción

Simular la asignación de CPU basada en ráfagas de tiempo.

Requisitos

- Implementar cola de prioridad con **min-heap**
- Cada proceso incluye:
 - ID
 - Tiempo de ráfaga

Condiciones

- Detectar:
 - **Overflow** (heap lleno)
 - **Underflow** (heap vacío)

Evidencia

- Inserciones y extracciones
- Manejo de errores

Análisis

- Justificación del modelo
- Complejidad de planificación

Ejercicio 5 — Optimización de Dijkstra con Min-Heap

Descripción

Optimizar el algoritmo de caminos mínimos.

Requisitos

- Implementar Dijkstra usando **min-heap**
- Soportar:
 - Inserción de nodos
 - Actualización de prioridad (**decrease-key**)

Evidencia

- Evolución de distancias
- Estado del heap en cada iteración

Análisis

- Comparación con implementación sin heap
- Mejora en complejidad:
 - De $O(n^2)$ a $O((n + m)\log n)$

6. Entregables

6.1 Documento técnico

Debe incluir:

1. Introducción
2. Explicación de heaps
3. Desarrollo por ejercicio:
 - Implementación
 - Evidencia
 - Análisis
4. Comparativa entre ejercicios
5. Conclusiones

6.2 Código fuente

- Implementación completa
- Estructura modular
- Comentarios claros

6.3 Evidencia

- Capturas o logs
- Pruebas realizadas
- Casos de uso

7. Criterios de evaluación

Criterio	Ponderación
Implementación correcta	30%
Uso adecuado de heaps	20%
Análisis y justificación	25%
Evidencia experimental	15%
Organización del código	10%

8. Restricciones

- No usar librerías predefinidas de heaps
- Implementación manual obligatoria
- Validar condiciones de error

9. Recomendaciones

- Implementar primero operaciones básicas
- Probar con casos pequeños
- Visualizar el heap como árbol
- Validar la propiedad de heap después de cada operación

10. Extensiones sugeridas

- Implementar heaps binomiales o Fibonacci
- Comparar diferentes métodos de prioridad
- Analizar rendimiento con grandes volúmenes de datos

11. Resultado esperado

Un conjunto de aplicaciones funcionales donde el estudiante:

- Comprenda la estructura de montículos
- Aplique heaps en problemas reales

- Analice su eficiencia y comportamiento